PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Raffi Maulana Abdillah

NPM 25782021

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

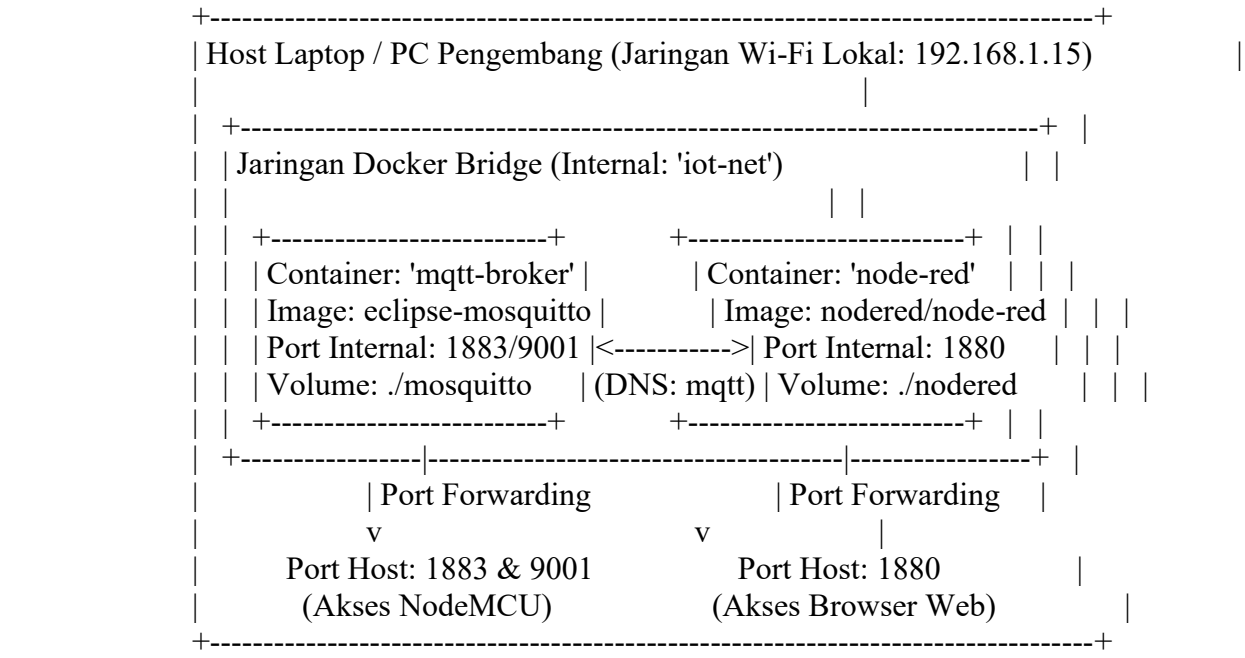
DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

PRAKTIKKUM 7

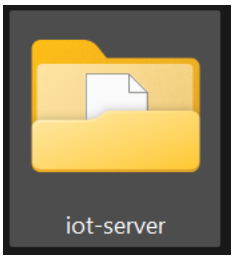
7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.



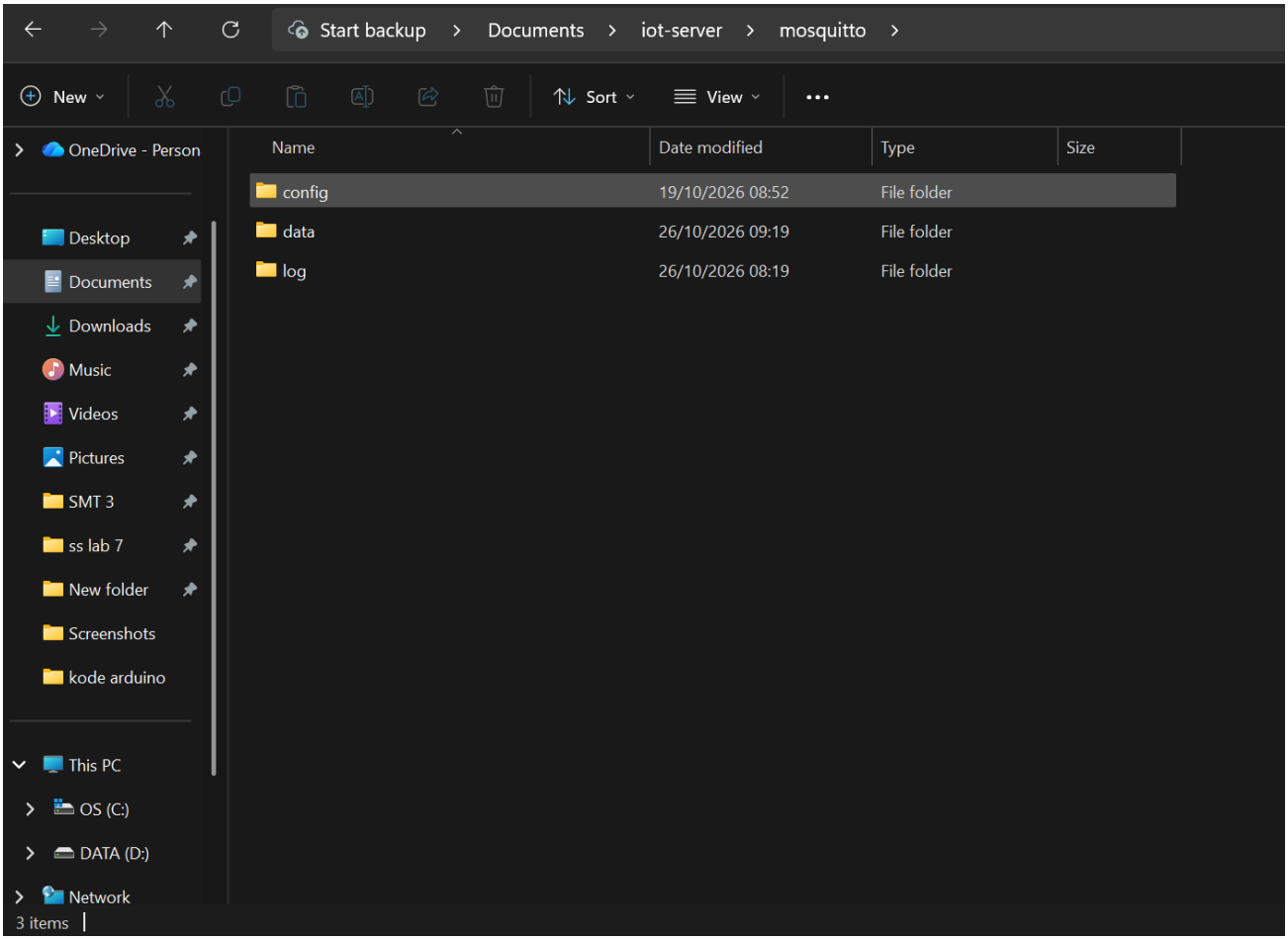
Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).



2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:

```
iot-server/
├── mosquitto/
│   └── config/
```



3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama **mosquitto.conf**, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:

```
#
================================================================================
=======
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
#
================================================================================
=======

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

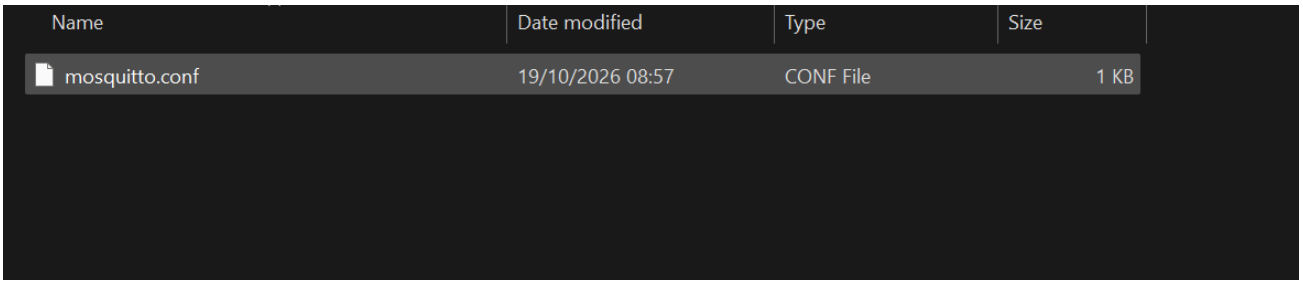
# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
```

```
persistence_location /mosquitto/data/

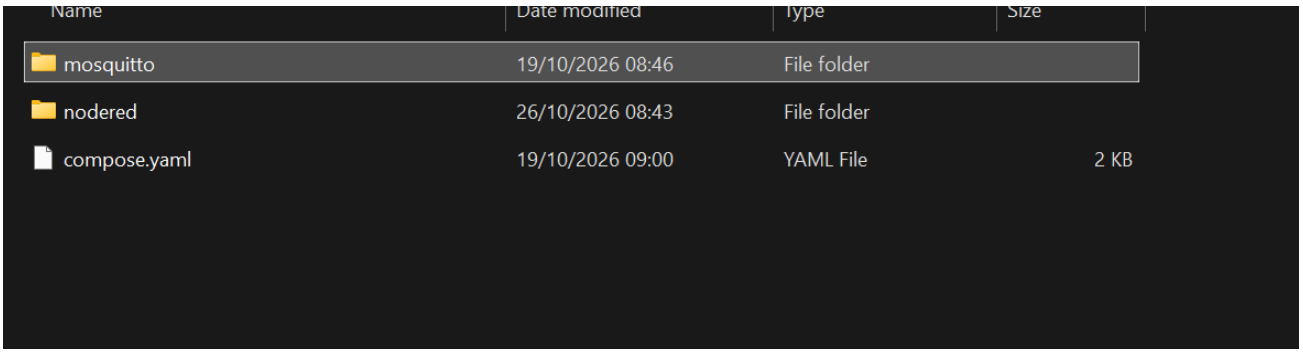
# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```



4. Kembali ke folder induk iot-server, buat berkas baru bernama **docker-compose.yml**, lalu salin konfigurasi layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```



5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:

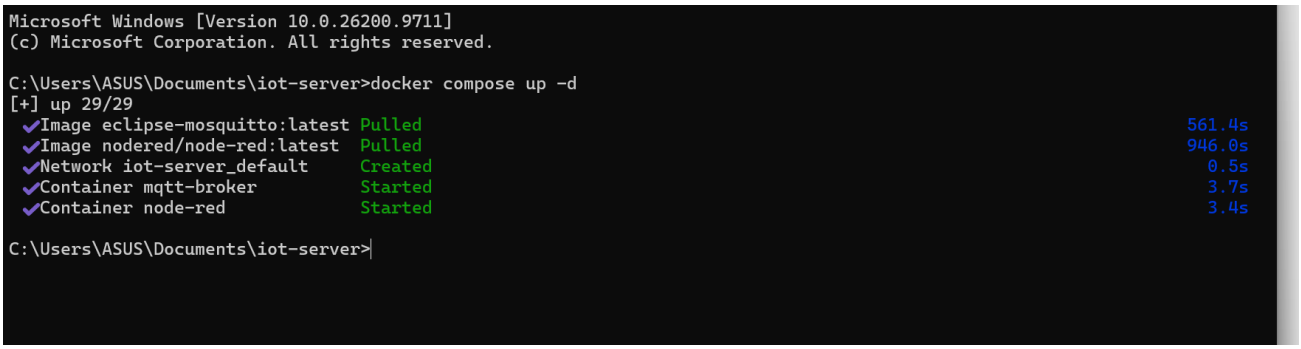
```
cd D:\iot-server
```

6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):

```
docker compose up -d
```

(Catatan: Pada versi Docker lama, gunakan perintah docker-compose up -d).

7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.



8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

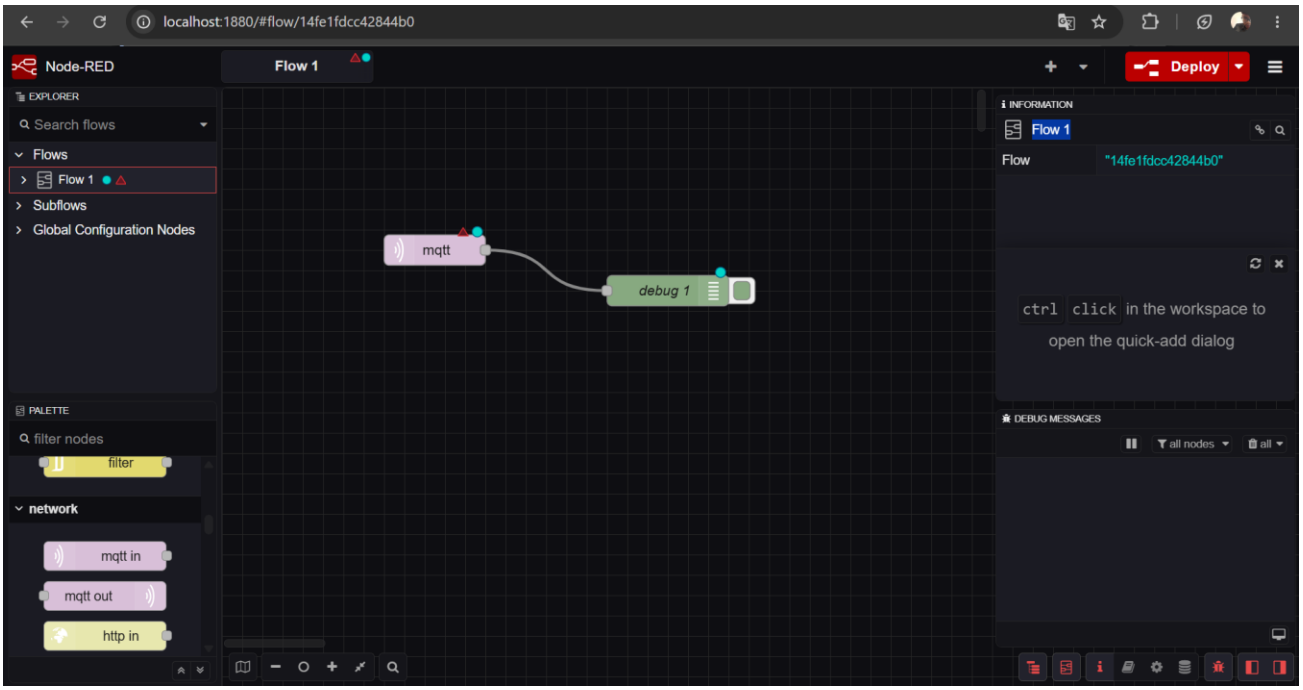
```
docker ps
```



Pastikan kolom STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).

9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: **http://localhost:1880**.

Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

Skema Rangkaian Perangkat Keras:

Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

- **Sensor DHT11/DHT22:**
  - Pin VCC →Pin 3V3 NodeMCU
  - Pin GND →Pin GND NodeMCU
  - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
  - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
  - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND

Langkah Pemrograman:

1. Cari tahu **Alamat IPv4 Lokal** laptop Anda yang terhubung ke jaringan Wi-Fi:
   - **Windows:** Buka CMD / PowerShell, ketik ipconfig. Temukan bagian *Wireless LAN adapter Wi-Fi*, lalu catat baris **IPv4 Address** (misalnya: 192.168.1.15).
   - **macOS / Linux:** Buka terminal, ketik ifconfig atau ip a (misalnya: 192.168.1.15).
2. **[PERINGATAN PENTING FIREWALL WINDOWS]:**
   Sistem keamanan *Windows Defender Firewall* sering kali secara otomatis memblokir koneksi masuk (*inbound connection*) dari port 1883. Jika NodeMCU gagal terhubung ke laptop, buka **Windows Defender Firewall with Advanced Security**, klik **Inbound Rules > New Rule**, pilih **Port > TCP**, masukkan port 1883, lalu pilih **Allow the connection**.
3. Buka **Arduino IDE**, buat berkas sketsa baru, lalu salin kode program terpadu berikut:

```
/*
 *
```

==========

```
=======
 * PRAKTIKUM 7: SIMPUL SENSOR-AKTUATOR IOT UNTUK BROKER LOKAL
 *
================================================================================
=======
 * Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
 * ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
 *
================================================================================
=======
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "NAMA_WIFI_HOTSPOT";
const char* password = "PASSWORD_HOTSPOT";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "192.168.1.15";
const int   mqttPort   = 1883;

// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT11 // Ganti DHT22 jika modul sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000; // Siklus kirim data setiap 4 detik

// Penanganan Paket Masuk dari Node-RED melalui Broker Lokal
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';
```

```
  Serial.println("\n----------------------------------------");
  Serial.printf("[INSTRUKSI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]        : %s\n", incomingBuffer);

  // Parsing JSON Instruksi Kendali: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);

  if (err) {
    Serial.printf("[ERROR] Gagal parsing JSON instruksi: %s\n", err.c_str());
    return;
  }

  // Evaluasi Kendali Indikator LED
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> [AKTUATOR] Status LED diubah: %s\n", stateLED == 1 ? "MENYALA
(ON)" : "PADAM (OFF)");
  } else {
    Serial.println("[WARNING] Kunci 'led' tidak ditemukan dalam payload!");
  }
  Serial.println("----------------------------------------");
}

void connectToWiFi() {
  Serial.println();
  Serial.print("[Wi-Fi] Menghubungkan ke ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[Wi-Fi] Berhasil Terhubung!");
  Serial.print("[Wi-Fi] IP NodeMCU: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
    Serial.print(mqttServer);
    Serial.print(")...");

    if (client.connect(clientId)) {
      Serial.println(" Terhubung!");
      // Segera daftarkan subskripsi topik kendali
```

```
      client.subscribe(topicSubCmd);
      Serial.printf("[MQTT] Berlangganan ke topik: %s\n", topicSubCmd);
    } else {
      Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Awal sistem: LED padam

  dht.begin();
  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageCallback);
  client.setBufferSize(512);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectToWiFi();
  }

  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Mendengarkan instruksi masuk

  // Pengiriman Telemetri Periodik Non-Blocking
  unsigned long now = millis();
  if (now - lastTeleTime >= teleInterval) {
    lastTeleTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      StaticJsonDocument<256> docOut;
      docOut["node_id"]    = clientId;
      docOut["suhu"]       = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);

      char teleBuffer[256];
      serializeJson(docOut, teleBuffer);
```

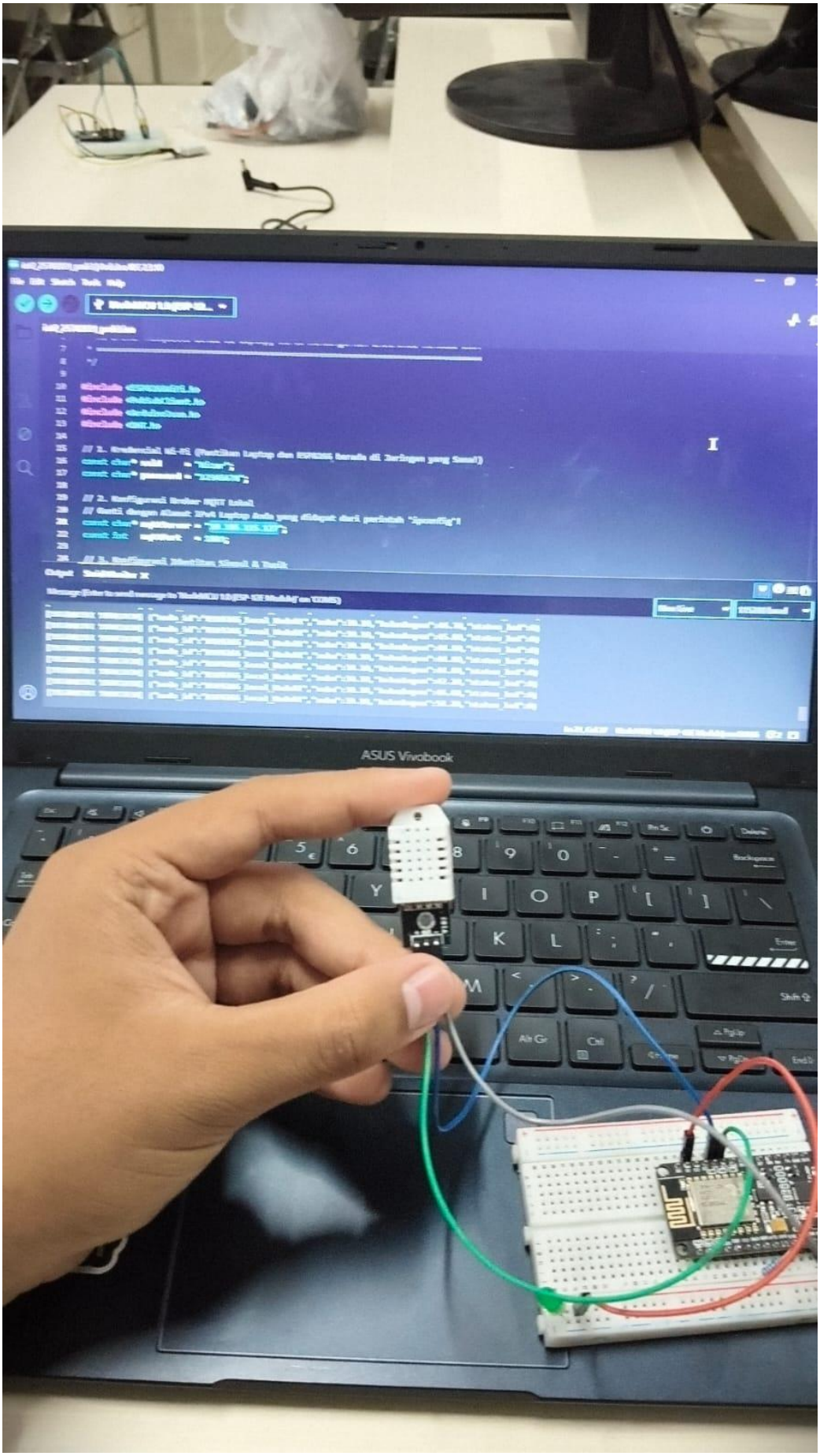
```
    bool success = client.publish(topicPubTele, teleBuffer);
    if (success) {
      Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
    } else {
      Serial.println("[ERROR] Gagal mengirim paket telemetri ke broker lokal!");
    }
  } else {
    Serial.println("[ERROR] Gagal membaca data dari sensor DHT fisik!");
  }
 }
}
```

4. Unggah (*Upload*) kode program ke board NodeMCU ESP8266.
5. Buka **Serial Monitor** pada kecepatan 115200 baud. Pastikan mikrokontroler berhasil mendapatkan alamat IP dan memunculkan tulisan [MQTT] Menghubungkan ke Broker Lokal... Terhubung! serta mulai mengirimkan payload telemetri JSON secara stabil.

KODE :

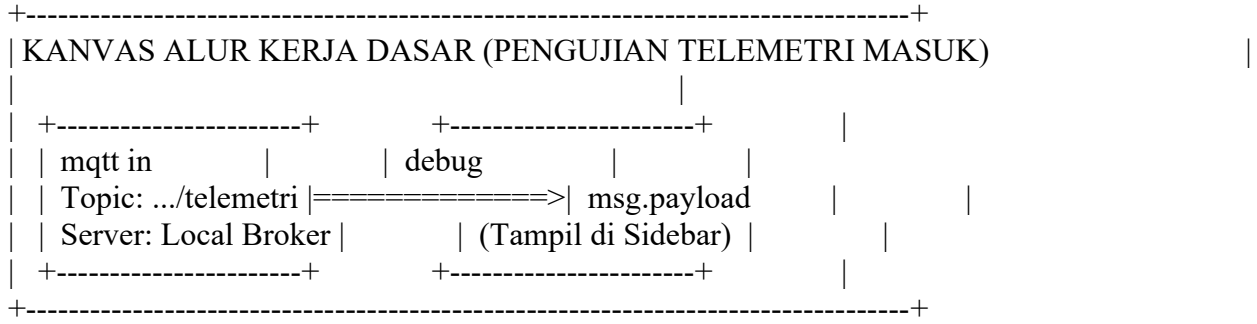


RANGKAIAN :


ASUS Vivobook

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

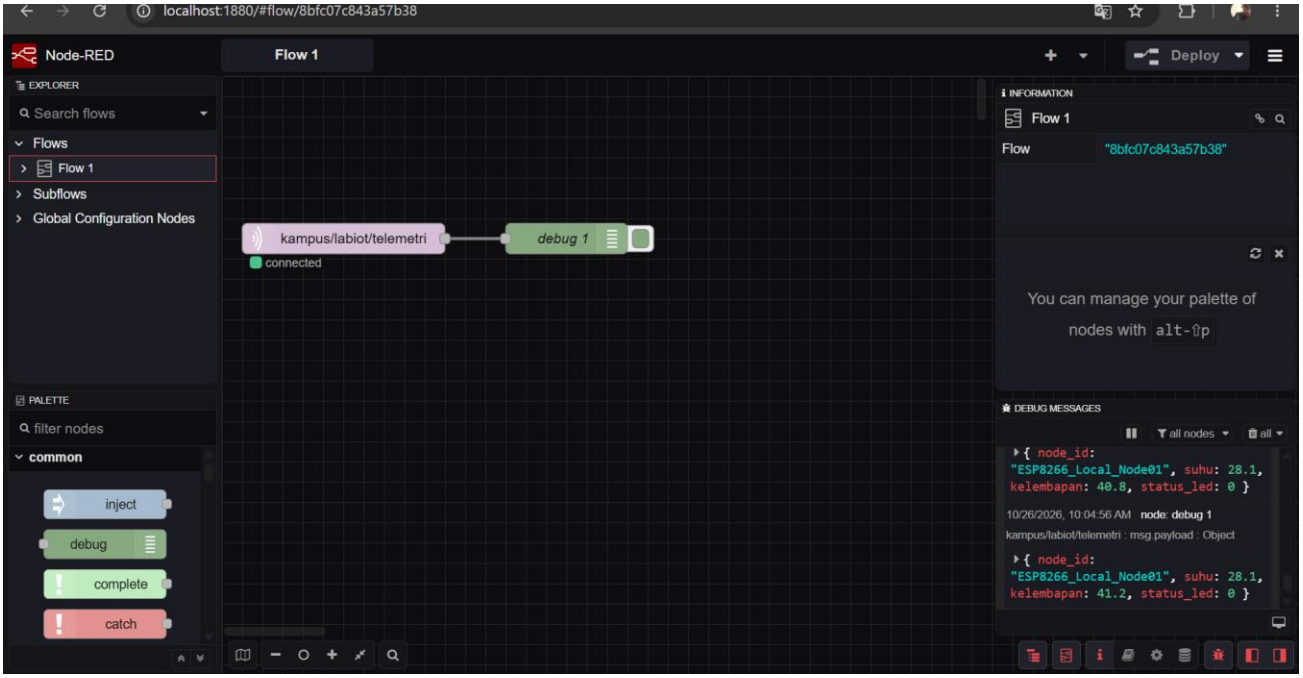


Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
   - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
   - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
     - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
     - **Name:** Mosquitto Local
     - **Server:** Ketik **mqtt**
       (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
     - **Port:** 1883
     - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
   - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
   - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
   - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
   - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
   - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**
   - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
   - Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.
6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang ( ) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:
   - kampus/labiot/telemetri : msg.payload : Object
   - {

- node_id: "ESP8266_Local_Node01",
- suhu: 29.8,
- kelembapan: 62.4,
- status_led: 0
- }
- Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.

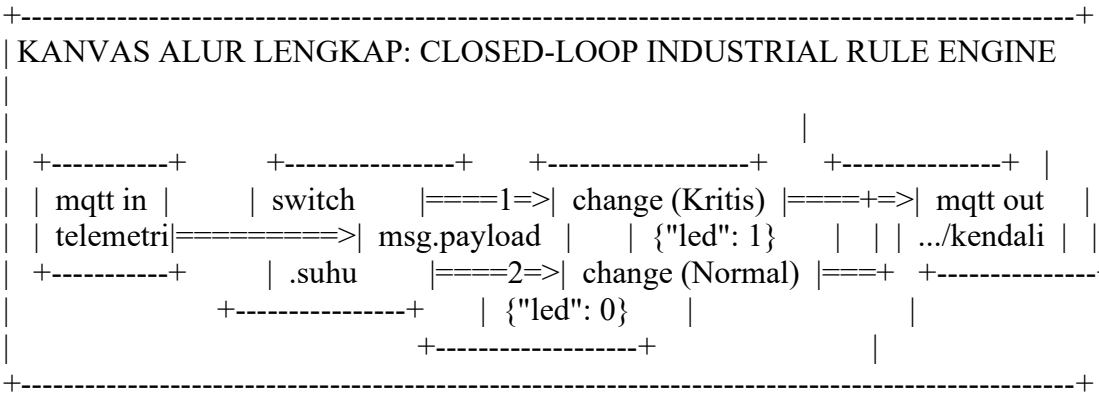
HASIL PRAKTIKKUM :



7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.



Langkah Kerja:

1. **Membuat Node Percabangan Logika (switch):**
   - Pada kategori **function**, seret node **switch** ke kanvas.

- Hubungkan port keluaran node mqtt in ke port masukan node switch.
- Klik ganda node switch untuk menyetel aturan logika evaluasi:
  - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
  - **Aturan 1 (Kondisi Panas):** Pilih operator **>**, lalu masukkan angka: **31**.
  - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
  - **Aturan 2 (Kondisi Aman):** Pilih operator **<=**, lalu masukkan angka: **31**.
  - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).

2. **Membuat Node Transformasi Muatan Panas (change):**
   - Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
   - Hubungkan port keluaran 1 (suhu > 31) dari node switch ke masukan node change pertama ini.
   - Klik ganda node change:
     - **Name:** Nyalakan LED Indikator
     - **Set:** msg.payload
     - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
     - {
     - "led": 1

}

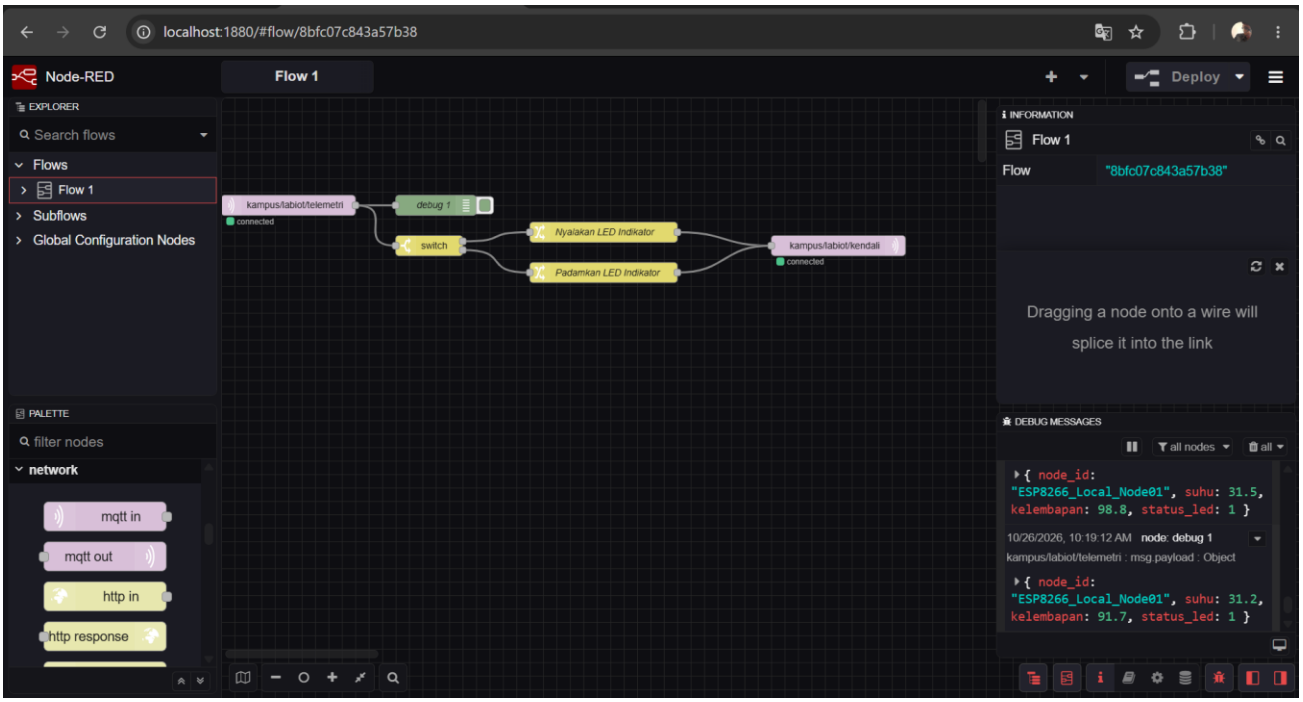
     - Klik **Done**.
3. **Membuat Node Transformasi Muatan Normal (change):**
   - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
   - Hubungkan port keluaran 2 (suhu ≤ 31) dari node switch ke masukan node change kedua ini.
   - Klik ganda node change:
     - **Name:** Padamkan LED Indikator
     - **Set:** msg.payload
     - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:
     - {
     - "led": 0

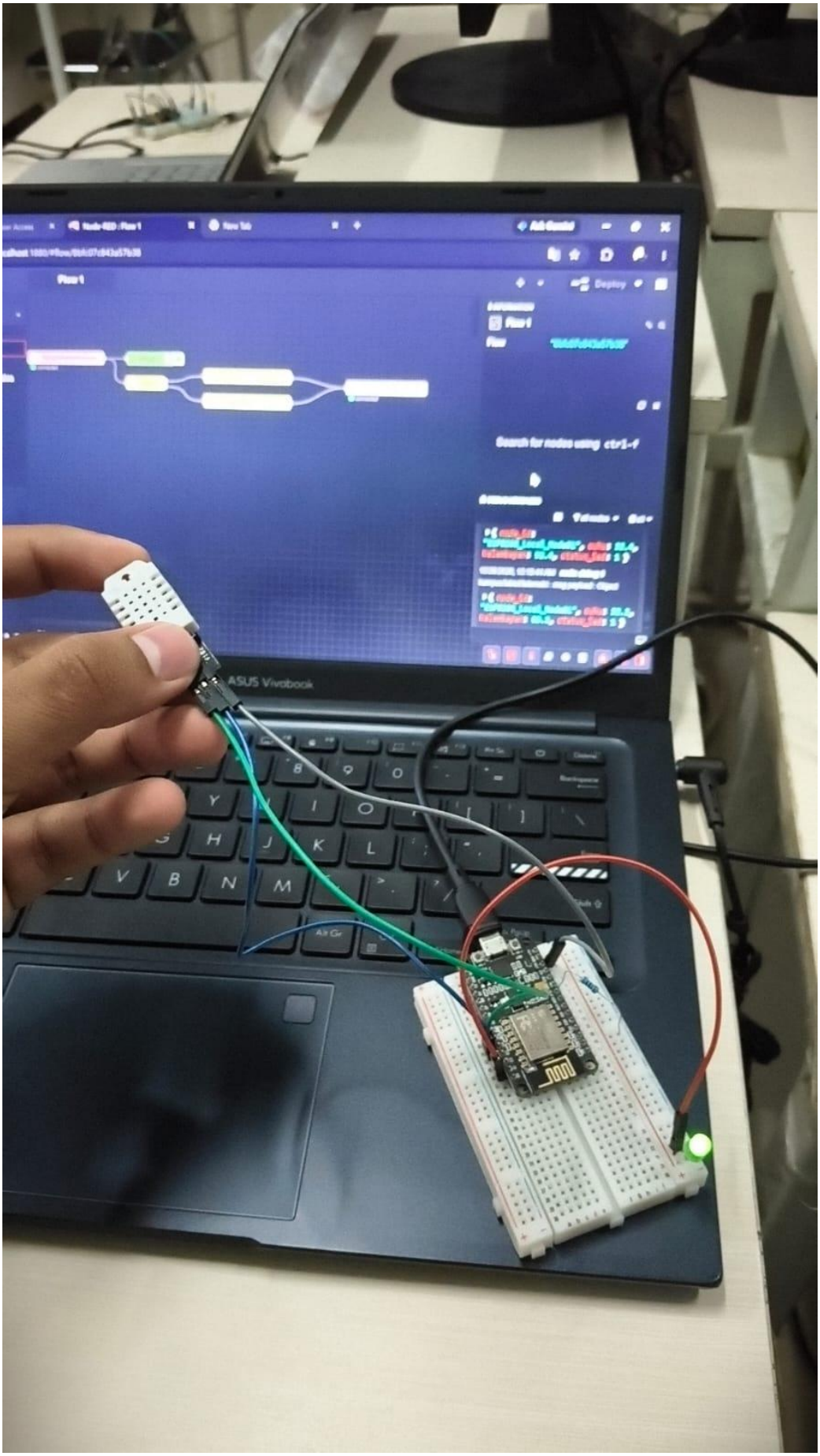
}

     - Klik **Done**.
4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
   - Pada kategori **network**, seret node **mqtt out** ke kanvas.
   - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
   - Klik ganda node mqtt out:
     - **Server:** Pilih Mosquitto Local.
     - **Topic:** kampus/labiot/kendali
     - **QoS:** 0 (atau 1)
     - **Retain:** false
     - Klik **Done**.
5. **Menerapkan Perubahan (*Deploy*):**
   - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**
   - Amati status awal: Karena suhu ruangan umumnya berkisar antara $26°C - 29°C (\leq 31°C)$, LED indikator berada dalam kondisi padam.
   - Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada

sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0°C.

- **Hasil Pengamatan:** Seketika suhu melampaui 31.0°C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
- Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0°C. LED indikator akan padam kembali secara otomatis.




ASUS Vivobook

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)
Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

- **Tujuan:** Mampu mengonfigurasi berkas kata sandi terenkripsi di dalam kontainer Mosquitto, menonaktifkan akses anonim, menganalisis dampak penolakan koneksi (*rc=5*), serta memperbarui konfigurasi klien ESP8266 dan Node-RED secara mandiri.
- **Tingkat Kesulitan:** Lanjut (Mandiri / Individu)

Instruksi Pengerjaan Langkah demi Langkah:
1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:
Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:
docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: **rahasia123** (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.
- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.

```
C:\Users\ASUS>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\ASUS>
```

2. Memperbarui Konfigurasi Kebijakan Mosquitto:
Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

- Ubah nilai allow_anonymous true menjadi:

allow_anonymous false

- Tambahkan baris pemanggilan berkas kata sandi:

password_file /mosquitto/config/password.txt

- Simpan berkas mosquitto.conf.

3. Memuat Ulang Layanan Broker:
Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:
docker restart mqtt-broker

```
C:\Users\ASUS>docker restart mqtt-broker
mqtt-broker

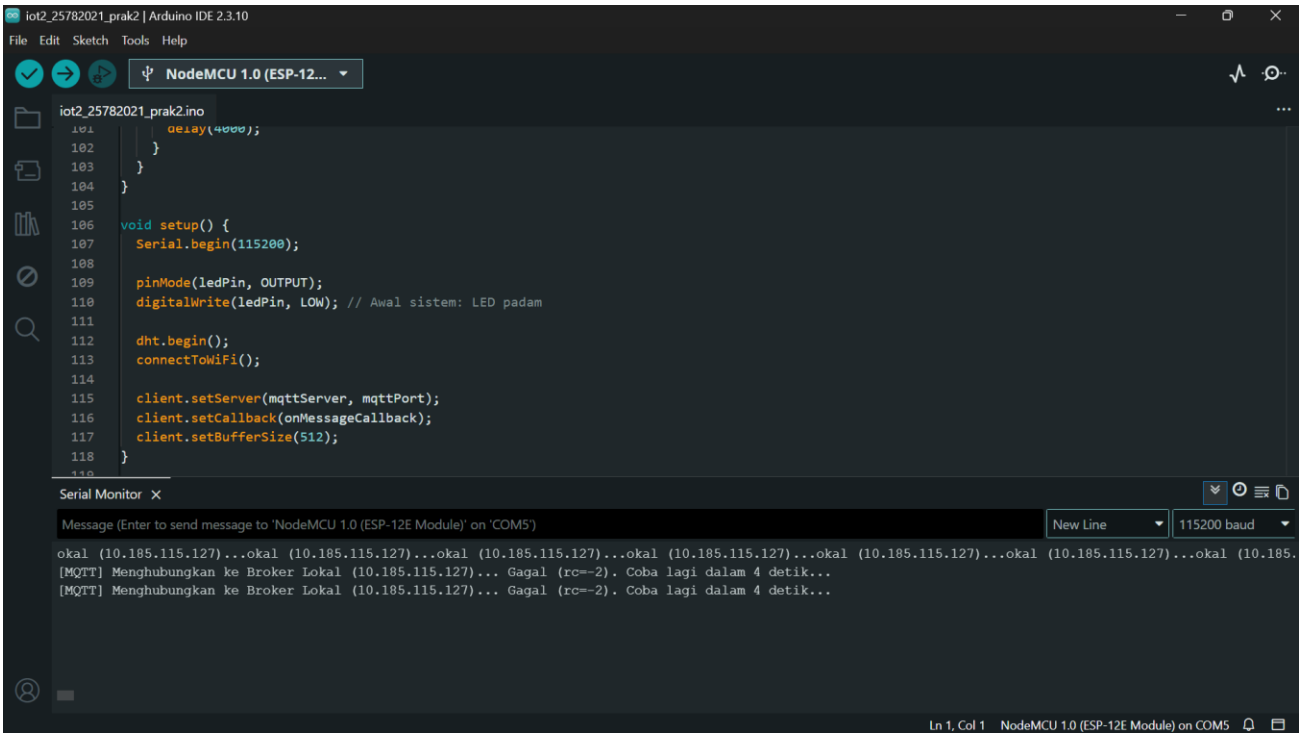
C:\Users\ASUS>docker restart mqtt-broker
mqtt-broker

C:\Users\ASUS>
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan **disconnected** atau **connection refused**.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:
- [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...

(Catatan: Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized').



5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

1. **Pemutakhiran Skrip Program NodeMCU ESP8266:**
    - Buka sketsa program Arduino IDE dari Praktikum 2.
    - Modifikasi bagian pemanggilan fungsi client.connect() di dalam fungsi reconnectMQTT() dengan menyertakan kredensial resmi:
    - // Format: client.connect(clientId, username, password)
    - if (client.connect(clientId, "admin", "rahasia123")) {
    - Serial.println(" Terhubung dengan Autentikasi Sah!");
    - client.subscribe(topicSubCmd);

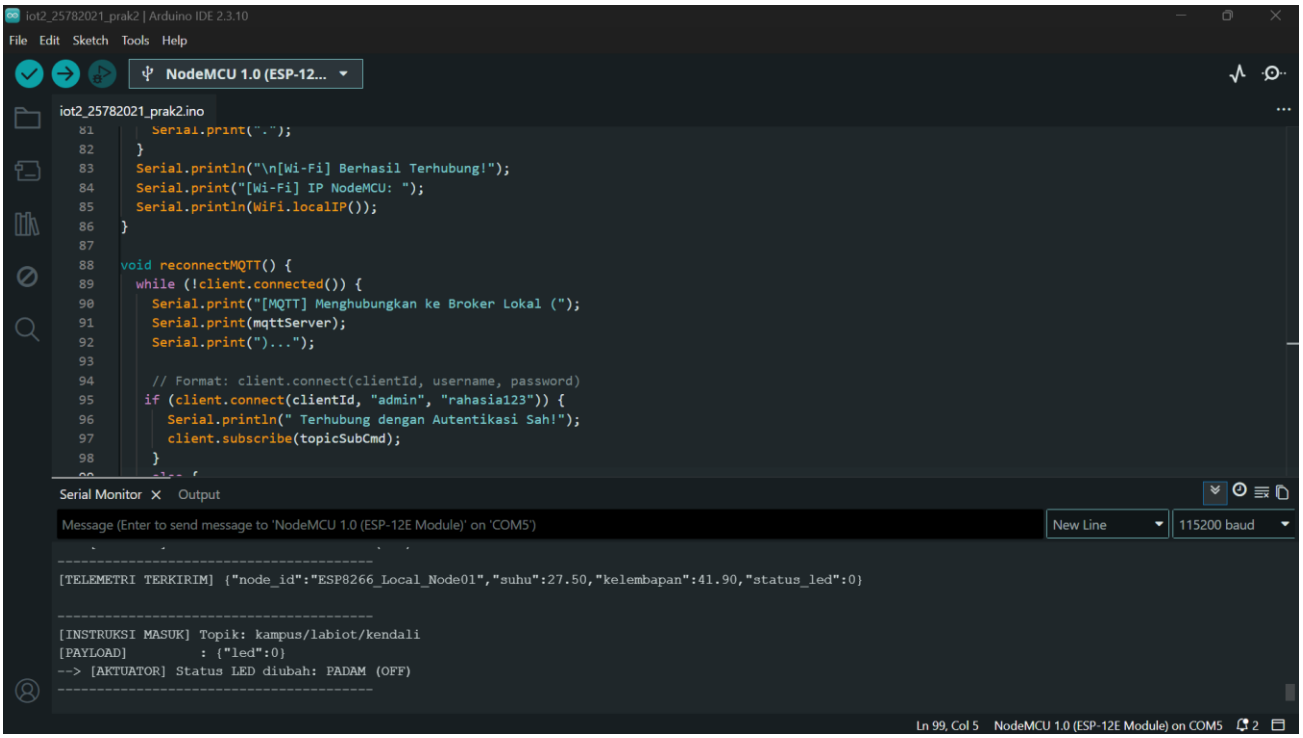
}

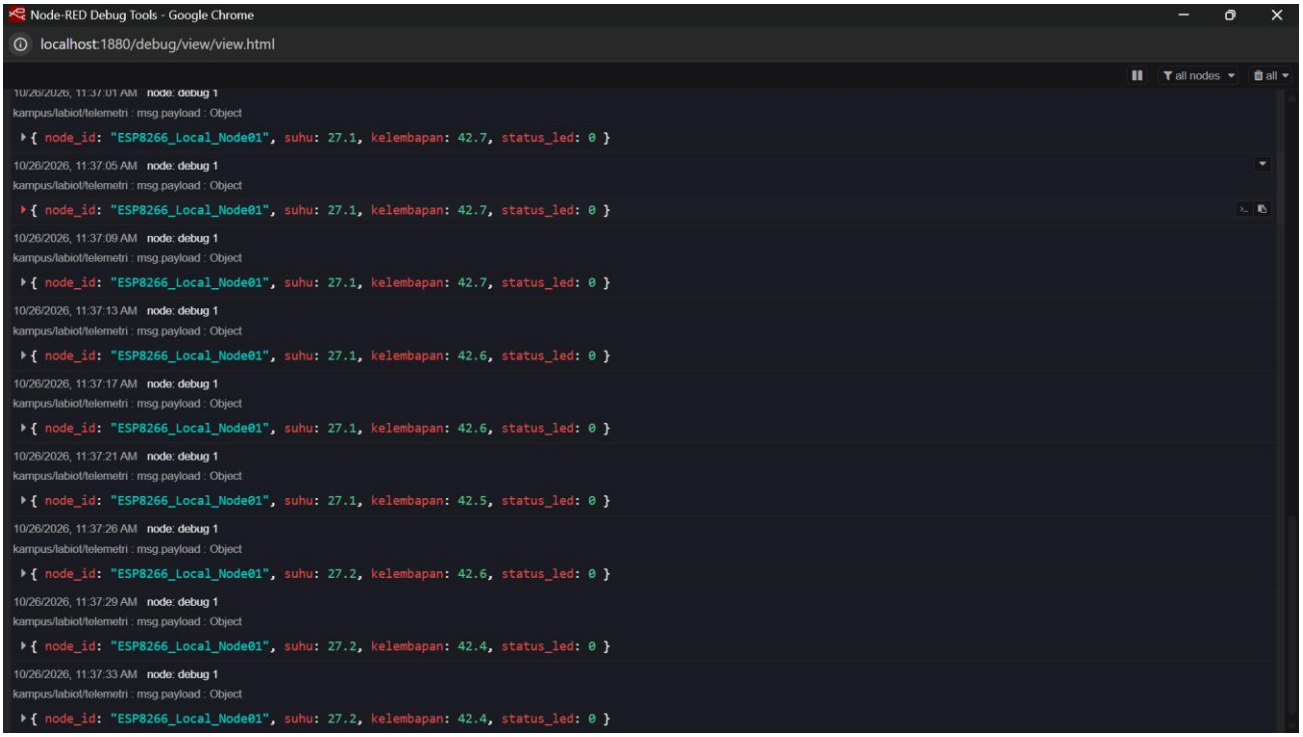
    - Unggah ulang kode ke NodeMCU dan amati bahwa ESP8266 kini kembali berhasil terhubung (*rc=0*).
2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**
    - Pada kanvas Node-RED, klik ganda node mqtt in.
    - Klik ikon pensil ( ) di samping nama broker Mosquitto Local.
    - Buka tab **Security** di bagian atas dialog konfigurasi.
    - Masukkan **Username:** admin dan **Password:** rahasia123.
    - Klik tombol **Update**, lalu klik **Done**, dan tekan tombol merah **Deploy**.
    - Amati bahwa indikator node MQTT pada kanvas Node-RED seketika kembali

berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar!

Kriteria Keberhasilan:

- Broker Mosquitto terbukti menolak akses anonim tanpa kredensial (allow_anonymous false).
- Terdapat berkas password.txt yang memuat enkripsi kata sandi akun pengguna.
- Serial Monitor NodeMCU membuktikan pemulihan koneksi sukses menggunakan kredensial autentikasi resmi.
- Seluruh aliran kerja (*flow*) pada Node-RED kembali terhubung dan mampu memicu otomasi LED secara stabil.




Node-RED Debug Tools - Google Chrome
localhost:1880/debug/view/view.html
all nodes
all
10/26/2026, 11:37:01 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.7, status_led: 0 }
10/26/2026, 11:37:05 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.7, status_led: 0 }
10/26/2026, 11:37:09 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.7, status_led: 0 }
10/26/2026, 11:37:13 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.6, status_led: 0 }
10/26/2026, 11:37:17 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.6, status_led: 0 }
10/26/2026, 11:37:21 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 42.5, status_led: 0 }
10/26/2026, 11:37:26 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.2, kelembapan: 42.6, status_led: 0 }
10/26/2026, 11:37:29 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.2, kelembapan: 42.4, status_led: 0 }
10/26/2026, 11:37:33 AM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.2, kelembapan: 42.4, status_led: 0 }